■■ AEGIS: Disaster & Emergency Response System

Complete DSA Viva & Project Defense Guide (Hinglish)

Subject: Data Structures & Algorithms (CS/IT) | **Architecture:** React + Leaflet + C++17/Python + SQLite

---

Project Summary: *AEGIS ek real-time disaster management and emergency decision support system hai. Ye natural disasters (jaise flood) ke time shortest safe rescue routes nikalta hai, distress calls ko priority score aur Max-Heap se rank karta hai, relief supplies ko 0/1 Knapsack Dynamic Programming se optimize karta hai, aur shelters me displaced victims ko Graph search aur BST/AVL structures ke through accommodate karta hai.*

1. Technology Stack & System Architecture

Component	Technology Used	Viva Explanation (Kyu use kiya?)
Frontend UI	React 18 + Vite + CSS	Single Page Application (SPA). Fast component rendering, real-time dynamic re-renders jab API response aaye.
Map & Visuals	Leaflet + React-Leaflet + OpenStreetMap	Interactive geospatial disaster map. Nodes ko markers aur edges/routes ko colored polyline roads ke roop me draw karta hai.
Core Backend	C++17 (Crow REST) + Python 3 Universal Engine	C++ ultra-low latency microsecond execution deta hai DSA algorithms ke liye. Python fallback guaranteed cross-platform execution deta hai.
Database	SQLite3 (Embedded Relational DB)	Zero-configuration, serverless, self-contained ACID compliant database. Graph nodes, roads, requests, teams, resources ko persist karta hai.

2. Page-By-Page Explanation & DSA Implementation

■ Page 1: Dashboard (Overview & Incident Command)

Screen par kya dikhta hai: Active disaster ka scenario ('Riverside Flood 2026', Severity 4), affected people count (2,340), KPI Stat Cards (Pending, Critical, Assigned, Resolved requests), Rescue Teams summary, Shelters occupancy progress bar, relief supplies category-wise, road blockage counter (2 out of 22 blocked), aur recent simulation event logs.
DSA & Technical Concept: Data Aggregation & Event Log Queue. SQLite database par relational aggregation queries (COUNT, SUM, GROUP BY) chalakar memory cache create ki jati hai. Simulation activity feed circular log queue format me maintain hoti hai.

■ Page 2: Disaster Map & Route Finder (Graph Algorithms)

Screen par kya dikhta hai: Interactive city map jisme 14 locations (bases, hospitals, relief camps, villages, bridges) aur 22 connecting roads hain. User Source aur Destination select karke algorithm (Dijkstra, BFS, DFS) choose karta hai aur **'Find Safe Route'** dabata hai. System safe route ko cyan color me highlight karta hai, distance aur microsecond execution time batata hai. Neeche table me kisi bhi road ko **'Block / Unblock'** toggle kar sakte hain.
DSA Implemented:
• **Graph Data Structure:** Weighted, Undirected Graph represented via **Adjacency List** (std::map<int, std::vector<Edge>>). Edges roads hain jinka weight distance (km) hai.
• **Dijkstra's Algorithm (Shortest Path):** Min-Heap (Priority Queue) ka use karke non-negative weighted graph me source se goal ka absolute shortest distance nikalta hai. Agar koi road flooded/blocked hai (is_blocked = 1), to algorithm us edge ko ignore kar deta hai.
• **Breadth-First Search (BFS):** Unweighted shortest path ya fewest hops/turns nikalta hai using FIFO Queue.
• **Depth-First Search (DFS):** Recursive stack traversal karta hai to check reachability.
• **Complexity:** Dijkstra: **O((V + E) log V)** time, **O(V + E)** space.

■ Page 3: Rescue Requests (Priority Queue, Heaps & Dispatch)

Screen par kya dikhta hai: Distress emergency requests ki table (Victim name, Emergency Type, Severity, People, Location, Waiting Time, Priority Score). Right side me interactive **Binary Max-Heap visualizer** jo priority order dikhata hai. 'Dispatch Team' click karne par nearest eligible rescue team assign ho jati hai.
DSA Implemented:
• **Multi-Factor Priority Scoring Formula:**
Score = SeverityWeight(10-50) + min(30, People*1.5) + min(20, Minutes*0.2) + TypeBonus(0-5)
Ye formula patient severity, crowd size, waiting latency, aur disaster category ko combine karke dynamic score deta hai.
• **Binary Max-Heap (Priority Queue):** Top priority request root par rehti hai. Max extraction **O(log N)** aur peek **O(1)** time me hoti hai.
• **Merge Sort:** Requests table ko sort karne ke liye Divide & Conquer Merge Sort ka use kiya gaya hai (Guaranteed **O(N log N)** stable sort).
• **Nearest Team Dispatch (Dijkstra):** Available rescue teams me se specialized eligibility check hoti hai (e.g., boat team for flood) aur road graph par Dijkstra chalakar nearest team dispatch hoti hai.

■ Page 4: Resource Optimizer (0/1 Knapsack Dynamic Programming)

Screen par kya dikhta hai: Relief supplies inventory (Food, Water, Medicine, First Aid, Rescue Equipment) with weights (kg) and utility value scores. User cargo truck ki capacity limit (e.g. 50 kg) daalta hai aur **'Run Optimization'** par click karta hai. Screen par **Dynamic Programming (Optimal)** aur **Greedy Heuristic** ka side-by-side comparison chart, DP matrix table, aur selected items list aate hain.

DSA Implemented:

• **0/1 Knapsack Problem:** Har emergency supply item ya to poora load hoga (1) ya chhod diya jayega (0). Fractional allowed nahi hota.

• **DP Recurrence Formula:**

DP[i][w] = max(DP[i-1][w], DP[i-1][w - wt[i]] + val[i]) (if wt[i] <= w)

• **Greedy vs DP Benchmark:** Greedy algorithm Value-to-Weight ratio (value / weight) se sort karta hai. Viva me explain karein ki Greedy 0/1 Knapsack me fail ho sakta hai kyu ki wo capacity waste chhod sakta hai, jabki DP hamesha mathematically optimal result deta hai.

• **Complexity:** DP Time: **O(N * W)**, Space: **O(N * W)**. Greedy Time: **O(N log N)**.

■ Page 5: Shelters & Victims Management (Trees, Hash Maps & Recommendation)

Screen par kya dikhta hai: Shelters ki list (Total capacity, current occupancy, status: Available / Limited / Full, medical facilities). Victims directory with instant real-time search. Victim ko nearest shelter recommend karne ka tool aur 1-click automatic assignment modal.

DSA Implemented:

• **Separate Chaining Hash Map:** Victim ID ke basis par instant **O(1) average lookup**. Collisions linked list chaining se resolve hote hain.

• **Binary Search Tree (BST) & AVL Self-Balancing Tree:** Victims ko ordered tree me store kiya jata hai. In-order traversal sorted list deta hai. AVL rotations (LL, RR, LR, RL) guarantee karte hain ki search time **O(log N)** se exceed na ho.

• **Singly Linked List:** Linear iteration and sequential traversal of victim records.

• **Smart Shelter Recommender (Dijkstra + Medical Match):** Victim ke location se sabhi open shelters ki distance nikalta hai. Agar victim ko medical assistance chahiye aur shelter me medical bay hai to 20 bonus score dekar best recommendation deta hai.

• **Atomic Transactions:** SQLite me BEGIN TRANSACTION ... COMMIT ke saath victim assignment aur shelter bed count ko sync rakhta hai.

3. Master DSA Complexity Summary Table (Viva Favorite)

Algorithm / Structure	Module	Time Complexity	Space Complexity	Primary Use Case
Dijkstra's Algorithm	Disaster Map	O((V + E) log V)	O(V + E)	Shortest safe road route avoiding flood/blocks
Breadth-First Search (BFS)	Disaster Map	O(V + E)	O(V)	Fewest hops route in unweighted road network
Depth-First Search (DFS)	Disaster Map	O(V + E)	O(V)	Road network connectivity & reachability check
Binary Max-Heap	Rescue Requests	Insert: O(log N) Extract: O(log N) Peek: O(1)	O(N)	Priority Queue for critical distress requests
Merge Sort	Rescue Requests	O(N log N) guaranteed	O(N)	Stable sorting of rescue requests by metrics
0/1 Knapsack DP	Resource Optimizer	O(N * W)	O(N * W)	Optimal relief cargo selection under weight limit
Greedy Knapsack	Resource Optimizer	O(N log N)	O(1)	Fast ratio-based heuristic comparison benchmark
Chaining Hash Map	Shelters & Victims	Avg: O(1) Worst: O(N)	O(N)	Instant O(1) victim ID lookup and indexing
AVL Balanced Tree	Shelters & Victims	O(log N) search/insert	O(N)	Height-balanced search tree preventing skewness

4. Top 20 Most Expected Viva Questions & Perfect Answers

Examiner aapse directly ye sawaal pooch sakta hai. Neeche exact answers diye gaye hain:

Q1. Aapke project ka naam kya hai aur ye kis problem ko solve karta hai?

Answer: Project ka naam **AEGIS (Advanced Emergency & Geospatial Intelligence System)** hai. Ye natural disasters (jaise flood) ke time decision-making automate karta hai. Disaster map par safe routes dhundhna, distress calls ko priority rank karna, relief truck me knapsack DP se essential supplies load karna, aur victims ko nearest shelter me assign karna iske main functions hain.

Q2. Disaster Map me graph ko kaise represent kiya gaya hai?

Answer: Graph ko **Adjacency List** (std::map<int, std::vector<Edge>>) ke through represent kiya gaya hai. Locations nodes hain (with lat/long coordinates) aur roads edges hain jinka weight distance (in km) hai. Adjacency List isliye choose ki kyunki road networks sparse graphs hote hain, jisme Adjacency Matrix space waste karti.

Q3. Map me shortest path ke liye Dijkstra hi kyu use kiya, BFS kyu nahi?

Answer: BFS sirf *unweighted* graph me shortest hops batata hai. Lekin roads ka distance alag-alag hota hai (weighted graph). Dijkstra non-negative weights ke sath absolute shortest distance nikalta hai. Isliye emergency me Dijkstra real shortest distance deta hai.

Q4. Flooded ya blocked roads ko algorithm kaise handle karta hai?

Answer: Database me roads table me is_blocked column hai. Graph load karte waqt ya Dijkstra traverse karte waqt blocked edge ko ignore (skip) kar diya jata hai ya uska weight infinity mana jata hai. Is tarah route hamesha safe roads se nikalta hai.

Q5. Rescue Requests me priority score kaise calculate hota hai?

Answer: Formula hai: Total = SeverityComponent(10-50) + PeopleComponent(max 30) + WaitingComponent(max 20) + TypeBonus(0-5). Critical condition ko 50 pts, per affected person 1.5 pts, per waiting minute 0.2 pts, aur Fire/Flood ko 5 bonus pts milte hain. Isse score 0 se 105 tak calculate hota hai.

Q6. Priority Queue ke liye Max-Heap kyu use kiya, normal array kyu nahi?

Answer: Unsorted array me maximum find karne me O(N) lagta hai aur sorted array me insertion me O(N) shift hota hai. **Binary Max-Heap** me insertion aur extraction dono **O(log N)** me hote hain aur maximum request **O(1)** me access hoti hai. Real-time emergency me Max-Heap optimal data structure hai.

Q7. Requests list ko sort karne ke liye Merge Sort kyu use kiya?

Answer: Merge Sort ek **Divide and Conquer** algorithm hai jo worst-case me bhi guaranteed **O(N log N)** time complexity deta hai (unlike Quick Sort jo worst case me $O(N^2)$ ho sakta hai). Sath hi Merge Sort **stable** hota hai, jisse same priority wale requests ka arrival sequence preserve rehta hai.

Q8. Resource Optimizer me 0/1 Knapsack problem kya hai?

Answer: Relief transport truck ki fixed weight capacity (W kg) hoti hai. Har emergency item (medicine, food, water) ka specific weight aur urgency value hota hai. 0/1 Knapsack ka goal hota hai ki capacity exceed kiye bina maximum utility value select ki jaye, jahan har item ya to poora include hoga ya exclude (fractional item allow nahi hota).

Q9. 0/1 Knapsack me Greedy Algorithm fail kyu ho sakta hai?

Answer: Greedy algorithm Value-to-Weight ratio (v/w) ke hisab se items chunta hai. Lekin 0/1 integer items me greedy approach truck me aisi empty capacity chhod sakti hai jisme doosra item fit na aaye. Dynamic Programming sabhi subproblems ko overlap karke optimal global solution deta hai, isliye 0/1 me DP superior hai.

Q10. Knapsack Dynamic Programming ki state definition aur recurrence relation kya hai?

Answer: State: DP[i][w] = pehle i items aur w capacity limit me maximum value. Recurrence: Agar item ka weight wt[i] <= w hai, to DP[i][w] = max(DP[i-1][w], DP[i-1][w - wt[i]] + val[i]), warna DP[i][w] = DP[i-1][w].

Q11. Shelters me victims ko store karne ke liye Hash Map kyu use kiya?

Answer: Disaster time par victim records ko instantly search karna hota hai. Hash Map victim ID ke hash code se bucket index nikal kar average **O(1) lookup time** provide karta hai. Collisions ko handle karne ke liye **Separate Chaining** (linked list at each bucket) use kiya gaya hai.

Q12. Binary Search Tree (BST) aur AVL Tree me kya difference hai?

Answer: Normal BST me agar data sorted order me insert ho to wo skew ho jata hai (worst-case search O(N) ho jati hai, jaise linked list). **AVL Tree** ek self-balancing BST hai jisme har node par Balance Factor (-1, 0, +1) maintain hota hai. Rotations (LL, RR, LR, RL) ke zariye height hamesha O(log N) rehti hai.

Q13. Shelter Recommendation engine kaise decide karta hai ki victim kis shelter me jaye?

Answer: Recommender engine candidate shelters filter karta hai jinki available capacity >= family members ho. Fir road graph par Dijkstra chala kar distance nikalta hai. Agar victim ko medical assistance chahiye aur shelter me medical bay hai to extra 20 score milta hai. Highest composite score wala shelter recommend hota hai.

Q14. Frontend aur Backend ke beech communication kaise hota hai?

Answer: React Frontend RESTful JSON API ke through Backend se communicate karta hai (endpoints jaise /api/dashboard, /api/map/route, /api/requests/dispatch). Vite proxy dev mode me port 5173 se port 8080 par requests forward karta hai.

Q15. Database consistency ke liye kya kiya gaya hai jab koi team dispatch ya victim assign hota hai?

Answer: Relational SQLite Database me **Atomic Transactions** (BEGIN TRANSACTION ... COMMIT) use kiye gaye hain. Jaise victim assign hone par: purane shelter se occupancy minus, naye shelter me add, aur victim ka status 'sheltered' teeno steps ek saath atomic unit me commit hote hain.

Q16. Project me C++ aur Python dono ka kya role hai?

Answer: Project ka core engine original C++17 me written hai for ultra-high performance DSA computations (Crow microframework ke sath). Local machine / security restriction (jaise Windows Device Guard) me chalane ke liye ek multi-threaded Python engine bhi banaya gaya hai jo exact same DSA logic aur endpoints serve karta hai.

Q17. Adjacency Matrix vs Adjacency List me kyu Adjacency List better hai is project ke liye?

Answer: Adjacency Matrix $O(V^2)$ space leti hai chahe edges kam ho. Hamare map me 14 locations aur 22 roads hain (sparse graph). Adjacency List sirf O(V + E) space leti hai aur neighbor exploration fast karti hai.

Q18. Kya aapke project me Leaflet Map offline kaam karta hai?

Answer: Haan, map coordinates, custom SVG icons, routes aur edge geometry local calculation se aati hai. Map tiles OpenStreetMap CDN se load hoti hain, lekin routing aur DSA algorithms 100% backend par offline calculate hote hain.

Q19. Space complexity ke terms me Knapsack DP ko kaise optimize kiya ja sakta hai?

Answer: Abhi 2D table O(N * W) use hoti hai backtracking ke liye. Agar sirf maximum value chahiye ho to 1D array DP[W] use karke space complexity O(W) me reduce ki ja sakti hai right-to-left iterate karke.

Q20. Is project ka real-world impact kya hai?

Answer: Natural disasters jaise flood, earthquake me human errors ko eliminate karta hai. Automation se shortest safe route seconds me milta hai, most critical victims pehle rescue hote hain, aur relief cargo optimally distribute hota hai jisse maximum lives save hoti hain.